

Trabajo Práctico Obligatorio (TPO)

Propuesta elegida: Propuesta 3

Sistema de Gestión Hotelera y Servicios para Huéspedes

Materia: Proceso de Desarrollo de Software

Docente: María Ángela León

Grupo: TeamVita

Integrantes:

Joaquín Manuel De Luca

Lautaro Vita

Claudino Diaz

Fecha de entrega: 20 de Junio de 2026

Repositorio Git: <https://github.com/joaacoo/TeamVita-sistema-hotelerero.git>

UADE

Fase 3:

1. Introducción y evolución del proyecto

Como fue establecido desde el inicio de este desarrollo, el diseño de nuestro Sistema de Gestión Hotelera no ha sido un proceso estático, sino una evolución continua. Durante la Fase 1, sentamos las bases identificando las entidades candidatas preliminares. Al avanzar hacia la Fase 2, sometimos ese modelo inicial a un riguroso análisis de diseño orientado a objetos. Esto nos permitió detectar fallas críticas de acoplamiento y cohesión, lo que derivó en la refactorización de la clase Reserva (extrayendo DetalleReserva para cumplir con el principio de Responsabilidad Única) e introduciendo el patrón Strategy en Huesped.

Para esta Fase 3, el enfoque se ha centrado en consolidar de forma práctica toda la infraestructura teórica plasmada en los diagramas UML anteriores. La transición del diseño al código ejecutable final demandó un proceso de mejora continua, incorporando validaciones de negocio reales como la verificación de huéspedes por DNI, la carga dinámica de acompañantes y la gestión de precios específicos para servicios adicionales desde la administración. Logramos así una correspondencia exacta entre la arquitectura en capas (Vista, Negocio, Model, Repo) y los componentes físicos del sistema, garantizando que cada patrón implementado responda a una necesidad real del negocio hotelero.

2. Descripción general de la problemática

Una cadena hotelera enfrenta dificultades para coordinar reservas, administrar distintos tipos de habitaciones y gestionar servicios personalizados. Gran parte de los procesos se encuentra distribuida en herramientas poco integradas o depende de operaciones manuales.

Problemas específicos identificados:

- Inconsistencias en la disponibilidad de habitaciones.
- Errores y demoras generados por la gestión manual de reservas.
- Falta de centralización de la información de huéspedes, reservas y habitaciones.
- Dificultad para gestionar reservas simultáneas en temporadas altas.
- Escasa automatización en los procesos de check-in y check-out.
- Dificultad para administrar promociones, descuentos y servicios personalizados.
- Poca integración entre los distintos servicios del hotel (spa, restaurante, limpieza, etc.).
- Dificultad para generar reportes y estadísticas de ocupación en tiempo real.

3. Objetivos del sistema

3.1 Objetivo general:

Desarrollar un sistema integral de gestión hotelera que permita administrar de manera centralizada reservas, habitaciones, huéspedes y servicios adicionales, aplicando principios de programación orientada a objetos, patrones de diseño y buenas prácticas de ingeniería de software.

3.2 Objetivos específicos

- Gestionar huéspedes, habitaciones y reservas del hotel.
- Permitir consultar disponibilidad de habitaciones en tiempo real.
- Automatizar procesos de check-in y check-out.
- Administrar servicios adicionales contratados por los huéspedes.
- Calcular costos finales considerando promociones, temporadas y descuentos.
- Generar reportes de ocupación e ingresos.
- Diseñar una arquitectura modular que permita agregar nuevas funcionalidades fácilmente.
- Permitir gestionar reservas que incluyan múltiples habitaciones.
- Permitir registrar pagos y señas asociados a reservas.

4. Alcance funcional

El sistema abarcará el ciclo completo de la estadía del huésped: desde la consulta de disponibilidad y registro de la reserva, pasando por la asignación de una o múltiples habitaciones, el registro de pagos o señas, el check-in, la asignación de servicios adicionales (spa, restaurante, lavandería), hasta el check-out con el cálculo automático del costo total de la estadía y la generación de la factura correspondiente.

Se incluirá también la gestión administrativa básica de habitaciones, huéspedes, reservas y reportes de ocupación.

El sistema será de uso interno del hotel, operado por recepcionistas y administradores.

5. Requisitos del sistema

5.1 Requisitos Funcionales (RF)

- **RF01:** El sistema debe permitir registrar, modificar y consultar los datos de los huéspedes.
- **RF02:** El sistema debe permitir administrar el catálogo de habitaciones (crear, modificar estado y categoría).
- **RF03:** El sistema debe permitir registrar, modificar y cancelar reservas.
- **RF04:** El sistema debe permitir consultar la disponibilidad de habitaciones filtrando por fechas y categoría.
- **RF05:** El sistema debe automatizar el registro de entrada (Check-in) y salida (Check-out) de un huésped.
- **RF06:** El sistema debe permitir asociar consumos y servicios adicionales a una estadía activa.
- **RF07:** El sistema debe calcular automáticamente el costo final de la estadía, aplicando las tarifas, promociones y recargos correspondientes.
- **RF08:** El sistema debe permitir registrar pagos o señas asociados a una reserva.
- **RF09:** El sistema debe permitir asociar múltiples habitaciones a una misma reserva.
- **RF10:** El sistema debe generar facturas correspondientes a una estadía finalizada.

5.2 Requisitos No Funcionales (RNF)

- **RNF01:** El sistema debe estar desarrollado íntegramente en lenguaje Java.
- **RNF02:** El diseño debe basarse en el paradigma de Programación Orientada a Objetos (POO).
- **RNF03:** La arquitectura debe aplicar al menos un patrón creacional, uno estructural y uno de comportamiento.
- **RNF04:** El diseño debe respetar al menos 3 principios SOLID y 3 patrones GRASP.
- **RNF05:** El control de versiones se realizará a través de un repositorio en GitHub.
- **RNF06:** El sistema deberá implementar persistencia de datos mediante una base de datos relacional MySQL.

- **RNF07:** El sistema contará con una interfaz gráfica desarrollada en Java Swing para la interacción con los usuarios.

6. Casos de uso y Actores

6.1 Identificación de actores:

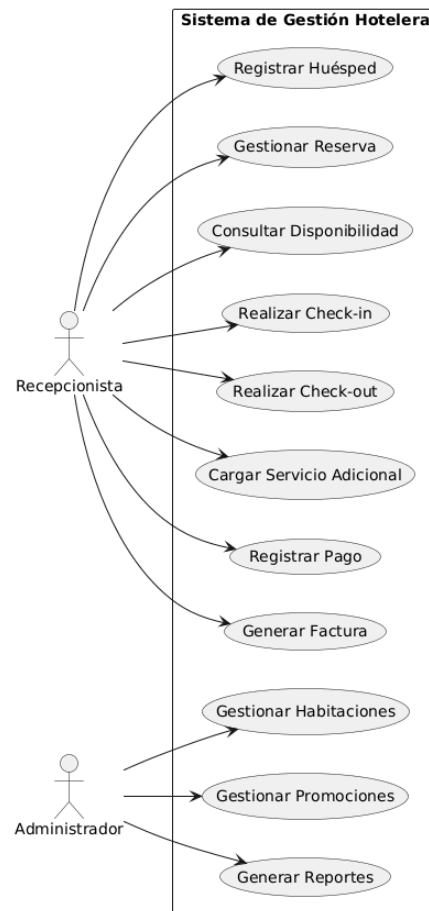
- **Recepcionista:** Usuario principal del sistema. Gestiona reservas, realiza el check-in/check-out y carga consumos a las habitaciones.
- **Administrador:** Usuario con privilegios extendidos. Configura tarifas, promociones, tipos de habitaciones y servicios, y visualiza reportes.

6.2 Casos de uso principales:

Casos de uso principales:

- **CU01 - Registrar Huésped:** Permite registrar, modificar y consultar la información de los huéspedes del hotel.
- **CU02 - Gestionar Reserva:** Permite crear, buscar, modificar o cancelar una reserva.
- **CU03 - Consultar Disponibilidad:** Permite verificar qué habitaciones están libres en un rango de fechas.
- **CU04 - Realizar Check-in:** Cambia el estado de la reserva y de la habitación cuando el huésped ingresa.
- **CU05 - Realizar Check-out:** Finaliza la estadía, emite el cálculo final a pagar y libera la habitación.
- **CU06 - Cargar Servicio Adicional:** Añade un cargo extra (ej. cena, spa) a la cuenta de una habitación ocupada.
- **CU07 - Registrar Pago:** Permite registrar una seña o pago asociado a una reserva.
- **CU08 - Generar Factura:** Permite emitir la factura final correspondiente a una estadía.
- **CU09 - Gestionar Habitaciones:** Permite al administrador crear, modificar estado y categoría de las habitaciones.
- **CU10 - Gestionar Promociones:** Permite configurar tarifas especiales y descuentos.

- **CU11 - Generar Reportes:** Permite visualizar estadísticas de ocupación e ingresos del hotel.



7. Arquitectura y Clases principales

7.1 Descripción de la arquitectura del sistema

El sistema ha sido completamente codificado en Java utilizando programación orientada a objetos pura. La estructura de paquetes se consolidó y refinó para asegurar el aislamiento de responsabilidades y un bajo acoplamiento:

- **com.teamvita.hotel.model (Capa de Dominio):** Aloja las entidades puras y las abstracciones del negocio. En esta fase se implementó la lógica interna completa de métodos clave como la asignación de estados polimórficos y la decoración de servicios
- **com.teamvita.hotel.negocio (Capa Lógica):** Contiene el controlador principal SistemaHotelFacade, el cual actúa como el cerebro coordinador de operaciones.
- **com.teamvita.hotel.repo (Capa de Persistencia):** Implementa la comunicación real con el motor de base de datos relacional a través del patrón Singleton en

ConexionBD y los respectivos Objetos de Acceso a Datos (DAOs).

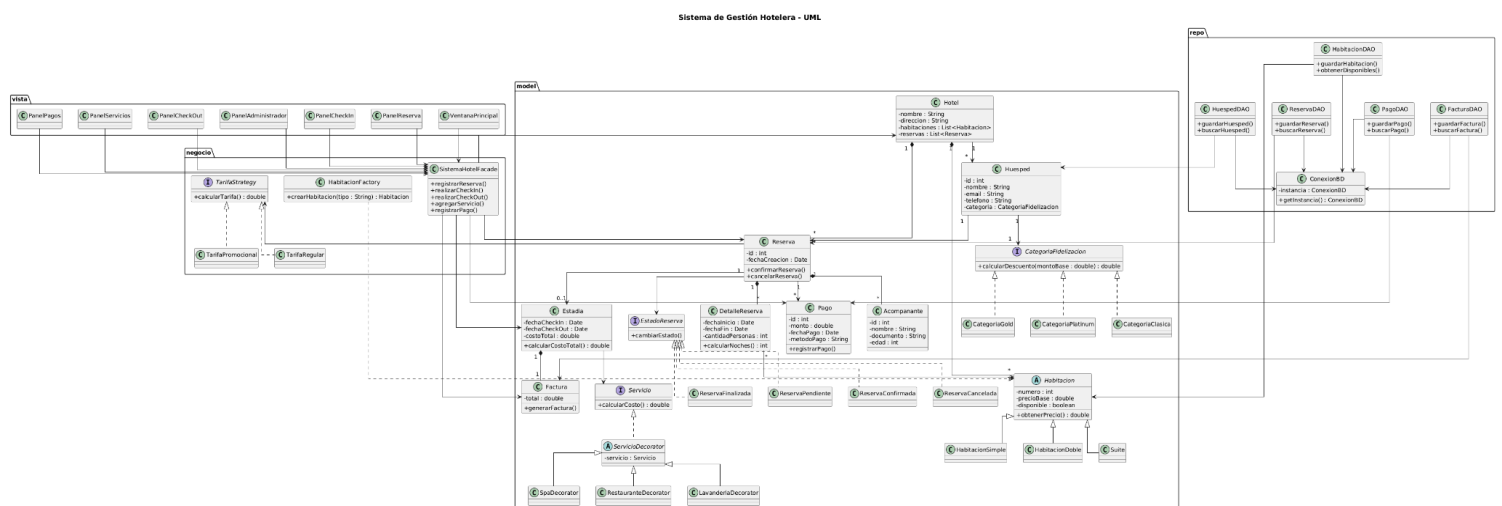
- **com.teamvita.hotel.vista (Capa de Presentación):** Compuesta por las ventanas e interfaces Swing (VentanaPrincipal, PanelReserva, PanelCheckIn, PanelAdministrador), que capturan los eventos del usuario y los delegan a la fachada.

7.2 Explicación de las clases principales

- **Hotel:** Actúa como la clase contenedora y gestora general de los recursos principales del sistema.
- **Huesped:** Almacena la información vital del cliente y gestiona su categoría de fidelización para interactuar con las reglas de negocio.
- **Estadia:** Representa el ciclo de vida real del alojamiento, desde el check-in hasta la facturación, vinculando los servicios consumidos y la habitación ocupada.
- **Reserva y DetalleReserva:** Manejan la intención previa de alojamiento, separando la parte administrativa (fechas, titular) de la especificación física (tipo de habitación).

8. Diagramas UML:

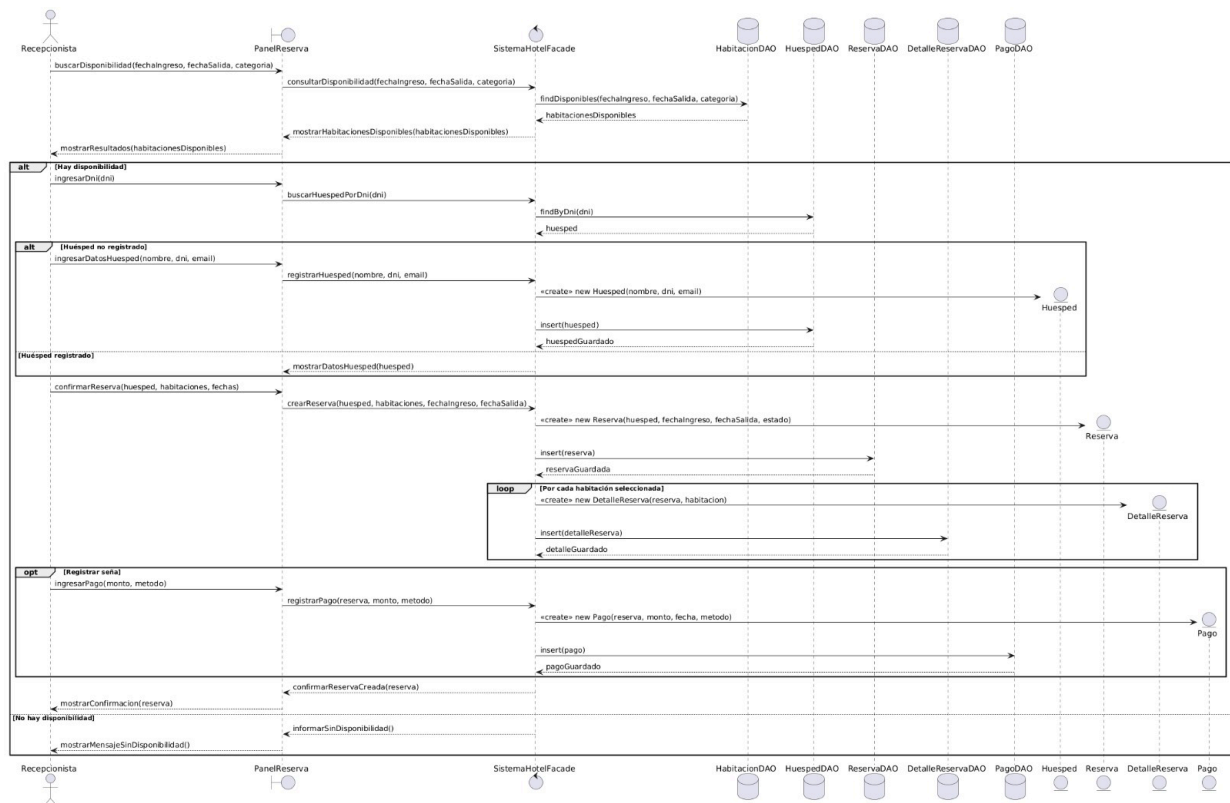
8.1 Diagrama de clases



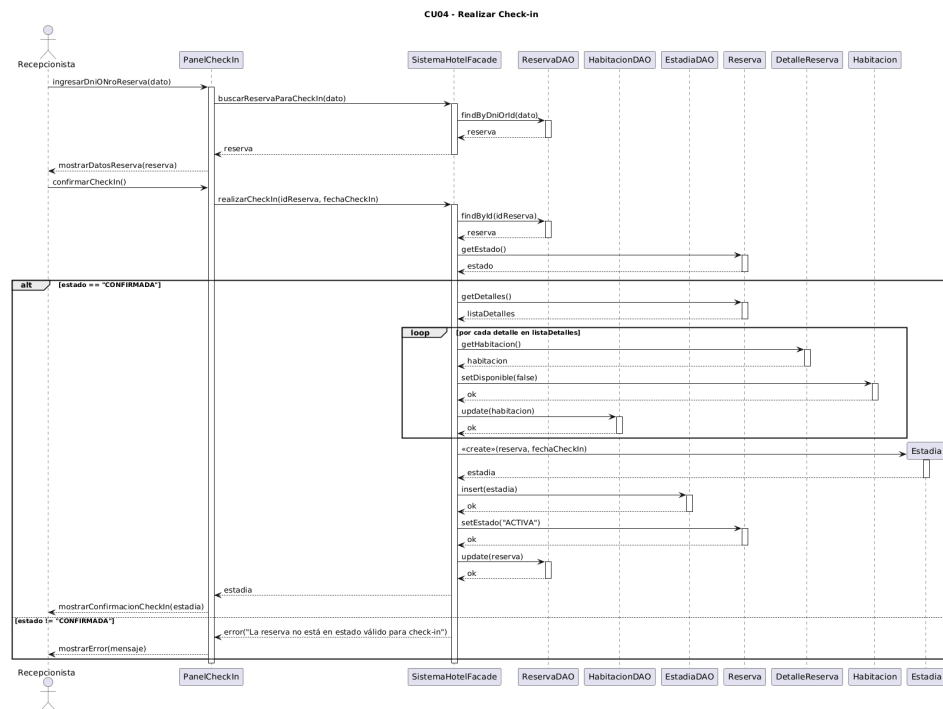
8.2 Diagrama de secuencia de casos de uso relevantes

En esta sección presentamos en detalle el funcionamiento de los dos casos de uso más representativos del sistema:

1. **CU02 - Gestionar Reserva (Creación):** El recepcionista interactúa con el PanelReserva. Éste delega a la clase SistemaHotelFacade la verificación de disponibilidad a través de HabitaciónDAO. Al confirmar disponibilidad y registrar/validar al Huesped (vía HuespedDAO), la Fachada instancia la Reserva y la persiste usando ReservaDAO. El estado inicial de la reserva se gestiona internamente al momento de la creación.



- CU04 - Realizar Check-in:** Desde el PanelCheckIn, se solicita el ingreso al hotel. SistemaHotelFacade recupera la Reserva con ReservaDAO. Se verifica su estado mediante el patrón State (EstadoReserva). Si el estado permite el ingreso (ej. ReservaConfirmada), se iteran las habitaciones asociadas, marcándolas como no disponibles a través del HabitacionDAO. Finalmente, se instancia la Estadia, se actualiza el estado de la reserva, y se persiste el cambio.



9. Identificación de patrones de diseño a aplicar y justificación

Para resolver la problemática planteada y garantizar una arquitectura flexible, se seleccionaron los siguientes patrones de diseño:

- Patrones creacionales:**

- Factory Method (HabitacionFactory):** Centraliza y encapsula la lógica de creación de las distintas variantes de habitaciones (Suite, HabitacionSimple, HabitacionDoble). Esto evita que la Fachada esté fuertemente acoplada a las clases concretas. De instanciar directamente con `new Suite()` en el código, la capa de vista o el controlador estarían fuertemente acoplados. El uso del Factory nos permite que el día de mañana el hotel agregue una nueva categoría (ej. 'HabitacionPresidencial') sin tener que tocar la lógica core de reservas.
- Singleton (ConexionBD):** Garantiza que exista una única instancia de conexión a la base de datos MySQL, optimizando el uso de recursos y

centralizando el punto de acceso para todos los DAOs en la capa repo.

- **Patrones estructurales:**

1. **Facade (SistemaHotelFacade):** Proporciona un punto de entrada unificado y simplificado desde el paquete vista hacia la lógica de negocio y model. Oculta la complejidad de interactuar con múltiples componentes del dominio y los repositorios de datos.
2. **Decorator (ServicioDecorator):** Permite añadir responsabilidades o servicios adicionales (SpaDecorator, RestauranteDecorator, LavanderiaDecorator) a la estadía de forma dinámica. Evita la proliferación de subclases para cada combinación posible de servicios contratados.

- **Patrones de comportamiento:**

1. **Strategy (TarifaStrategy):** Encapsula los algoritmos de cálculo de precios (TarifaPromocional, TarifaRegular, TarifaCorporativa, TarifaTemporadaAlta), permitiendo cambiar la política comercial en tiempo de ejecución sin alterar el código de la reserva o estadía. Sin este patrón, la clase Estadía tendría múltiples bloques ifs para calcular costos. Cada vez que marketing lanzara una nueva promoción, tendríamos que modificar el código central del sistema, violando el principio Open/Closed.
2. **State (EstadoReserva):** Administra las transiciones lógicas de la reserva (ReservaPendiente, ReservaConfirmada, ReservaCancelada, ReservaFinalizada). Delega el comportamiento dependiente del estado a clases concretas, eliminando bloques condicionales extensos y propensos a errores.
3. **Strategy (CategoriaFidelizacion):** Se aplica este patrón en la clase Huesped para encapsular el algoritmo de cálculo de descuentos según su nivel de membresía (Clásica, Gold, Platinum). Esto le otorga al huésped un rol activo en las reglas de negocio y permite asignar beneficios dinámicos en la facturación sin llenar el código de sentencias condicionales.

9.1 Patrones SOLID:

1. **Single Responsibility Principle (SRP):** Cada clase del sistema tiene una única responsabilidad bien definida. Las clases del paquete model (como Huesped o Reserva) se encargan exclusivamente de representar el dominio, mientras que las clases del paquete repo (como FacturaDAO) son las únicas responsables de interactuar con la base de datos. El ejemplo más claro de la aplicación de este principio fue la refactorización de la clase Reserva, la cual fue dividida en Reserva (responsable del ciclo de vida y datos administrativos) y DetalleReserva (responsable de la especificación de la habitación, capacidad y fechas exactas de

ocupación). Esto evitó la creación de un anti-patrón de 'Clase Dios'.

2. **Open/Closed Principle (OCP):** La arquitectura está diseñada para ser abierta a la extensión pero cerrada a la modificación. Gracias al patrón Decorator, se pueden incorporar nuevos servicios adicionales al hotel sin necesidad de modificar el código existente en las clases de facturación o estadía.
3. **Liskov Substitution Principle (LSP):** Las clases derivadas pueden sustituir a sus clases base sin alterar el correcto funcionamiento del sistema. Esto se evidencia claramente en la jerarquía de habitaciones: en cualquier proceso que requiera operar con un objeto del tipo base Habitación (por ejemplo, al calcular el precio base), se le puede suministrar una instancia de Suite, HabitaciónSimple o HabitaciónDoble, y el sistema responderá correctamente a través del polimorfismo.
4. **Dependency Inversion Principle (DIP):** El modelo de dominio cumple fuertemente con este principio al depender de abstracciones (interfaces) y no de implementaciones concretas. Por ejemplo, en lugar de que la clase Reserva o Estadía dependa directamente de una clase concreta como TarifaTemporadaAlta, depende de la interfaz pura TarifaStrategy. De esta forma, el módulo de alto nivel (el cálculo de la estadía) invierte su dependencia hacia una abstracción, permitiendo agregar nuevas lógicas de precios sin alterar el código que las consume. Lo mismo ocurre con la interfaz EstadoReserva.
5. **Interface Segregation Principle (ISP):** Se diseñaron interfaces pequeñas y de propósito único, en lugar de crear "Interfaces Dios". Un claro ejemplo es la interfaz TarifaStrategy, la cual expone únicamente el método calcularTarifa(). De este modo, cualquier clase cliente o nueva estrategia que se implemente no se ve forzada a depender o implementar métodos que no necesita.

9.2 Patrones GRASP aplicados:

1. **Controlador (Controller):** La clase SistemaHotelFacade actúa como el controlador principal que recibe, coordina y delega las operaciones del sistema solicitadas por la interfaz gráfica.
2. **Experto en Información (Information Expert):** La clase Estadía posee la responsabilidad de calcularCostoTotal() ya que es la experta en información: es la única que tiene acceso a los días alojados, la reserva asociada y los servicios adicionales consumidos por el huésped.
3. **Alta Cohesión y Bajo Acoplamiento:** La separación estricta en capas (vista, negocio, model y repo) garantiza que cada paquete trabaje con elementos estrechamente relacionados (alta cohesión) minimizando las dependencias directas con el funcionamiento interno de las demás capas (bajo acoplamiento).
4. **Polimorfismo (Polymorphism):** Se asignan responsabilidades utilizando operaciones polimórficas para manejar el comportamiento que varía según el tipo. Esto es clave en el sistema y se evidencia al usar las interfaces TarifaStrategy,

EstadoReserva y CategoriaFidelizacion. En lugar de tener una clase central con múltiples if/switch para saber cómo calcular un descuento o validar un cambio de estado, el sistema delega la ejecución al objeto polimórfico correspondiente.

5. **Fabricación Pura (Pure Fabrication):** Para no violar la Alta Cohesión y el Bajo Acoplamiento, se crearon clases que no representan conceptos del dominio real del problema (en un hotel físico no existe un "DAO" o una "Conexión a BD"). Las clases SistemaHotelFacade, ConexionBD, HabitacionFactory y todos los DAOs son Fabricaciones Puras introducidas artificialmente en el diseño de software para organizar el código y aislar responsabilidades.
6. **Creador (Creator):** Se aplica este patrón al asignar la responsabilidad de instanciar los objetos complejos. Por ejemplo, el SistemaHotelFacade actúa como Creador de la Reserva y la Estadia, ya que es la clase que recolecta y posee los datos de inicialización enviados desde la interfaz gráfica (Panel de Recepción) y tiene la responsabilidad de registrar esa instancia en la base de datos a través del DAO.
7. **Indirección (Indirection):** Para evitar el acoplamiento directo entre la capa de Presentación (Vista) y la capa de Dominio o Persistencia, se introdujo el SistemaHotelFacade. Esta fachada actúa como un componente intermediario (indirección). Gracias a esto, un PanelReserva no interactúa directamente con HabitacionDAO o con la entidad Reserva, sino que le envía el mensaje a la Fachada, logrando un bajo acoplamiento.
8. **Variaciones Protegidas (Protected Variations):** El sistema está diseñado para que las variaciones y la inestabilidad en ciertas lógicas no afecten al resto del sistema. Al encapsular el cálculo de tarifas detrás de la interfaz TarifaStrategy, el núcleo de la Estadia queda "protegido". Si mañana el hotel decide crear una nueva política de precios (una variación), el sistema no se rompe ni requiere modificar el código existente; simplemente se crea una nueva clase que implemente la interfaz.

10. Pruebas y evidencia Visual

Para asegurar que el software cumple con el flujo integral (Problema → Análisis → Diseño → Código → Prueba), se diseñaron y ejecutaron los siguientes escenarios críticos en la aplicación:

Escenario 1: Gestión de reserva y carga de huésped

- **Entrada:** El recepcionista abre el PanelReserva e ingresa los datos de un nuevo huésped.
- **Procesamiento:** Al ingresar el DNI, se recupera el perfil del cliente y se autocompleta su categoría de fidelización mediante el patrón Strategy.
- **Resultado esperado:** La interfaz gráfica refleja inmediatamente el beneficio comercial asociado al tipo de cliente.

TeamVita - Sistema de Gestión Hotelera - Usuario: recepcion

Nueva Reserva | Gestionar Reservas | Gestión de Huéspedes

Registrar Nueva Reserva

DNI Huésped (Máx 8):

Nombre Completo:

Correo Electrónico:

Teléfono:

Categoría Fidelización: ▼

Check-In: ...

Check-Out: ...

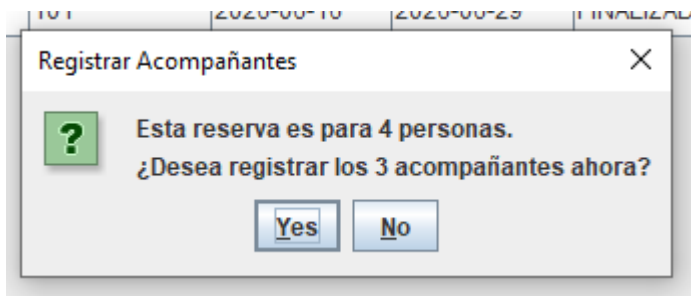
Habitación Disponible: ▼

Cantidad de Personas: ▲ ▼

Promoción: ▼

Escenario 2: Formulario dinámico de acompañantes

- **Entrada:** En la pantalla de reservas, el usuario define la "Cantidad de personas" (por ejemplo, 3 personas).
- **Procesamiento:** La vista genera dinámicamente el formulario de acompañantes para evitar errores de carga manual y mejorar la experiencia de usuario (UX).
- **Resultado esperado:** Al haber seleccionado 3 pasajeros, el sistema despliega exactamente 2 filas adicionales para registrar a los acompañantes obligatorios.



Escenario 3: Carga dinámica de consumos extra (Patrón Decorator)

- **Entrada:** El recepcionista abre la ventana de "Añadir consumo extra" para un servicio adicional (ej. Spa o Restaurante).
- **Procesamiento:** Al seleccionar el tipo de servicio desde el menú desplegable, el sistema consulta en tiempo de ejecución a la base de datos a través de ServicioDAO.
- **Resultado esperado:** El campo "Precio unitario" se actualiza instantáneamente con el valor real configurado en la base de datos, demostrando la eliminación de subclases estáticas a favor del decorador dinámico.

Escenario 4: Check-out y generación del ticket de facturación final

- **Entrada:** El huésped finaliza su estadía y se procede al cierre administrativo del caso de uso.
- **Procesamiento:** SistemaHotelFacade invoca la lógica de facturación de Estadía. Se calcula el costo base aplicando la TarifaStrategy, se suman los consumos del decorador, se resta la seña previa y se aplica el descuento por fidelización.
- **Resultado esperado:** El sistema despliega un cuadro de diálogo con el desglose pormenorizado y el total neto exacto a abonar.

Check-Out y Facturacion - Joaco

--- FACTURA DE ESTADIA ---

Huesped: Joaco

Total Habitación:	\$2250,00
Servicios Extra:	\$80,00
Total de estadía:	\$2330,00
Senia ya abonada:	-\$675,00

SALDO A COBRAR: \$1655,00

Medio de pago del saldo:

Factura Emitida

i CHECK-OUT COMPLETADO

Huesped: Joaco

Total cobrado: \$2330,00

Medio de pago del saldo: Efectivo

Gracias por su estadía!

Escenario 5: Panel de administración y reportes en tiempo real

- **Entrada:** El usuario con privilegios extendidos (Administrador) ingresa al PanelAdministrador.
- **Procesamiento:** Para cumplir con los casos de uso principales de gestión, el controlador consolida la información consultando a los DAOs correspondientes (HabitacionDAO, FacturaDAO).
- **Resultado esperado:** El sistema despliega el Dashboard de Administración mostrando métricas en tiempo real (porcentaje de ocupación, habitaciones libres, ingresos totales) y habilita la grilla para crear o editar las tarifas base del catálogo de habitaciones.

TeamVita - Sistema de Gestión Hotelera - Usuario: admin

Administración

Dashboard de Administración

Ocupación Actual
0%

Habitaciones Libres
3

Ingresos Totales
\$ 4400,00

Habitaciones y Tarifas | Servicios Extra y Config. | Promociones

Gestión de Habitaciones y Tarifas

Número	Tipo	Precio Base	Capacidad	Disponible
101	Simple	\$ 50,00	1	Sí
102	Doble	\$ 80,00	2	Sí
201	Suite	\$ 150,00	6	Sí

Nueva Habitación

Editar Habitación

11. Integración y reflexión

11.1 Registro de avance e integración individual

- **Joaquín De Luca:** Consolidó la codificación final de las entidades del paquete model (Hotel, Huesped, Reserva, Estadia, etc.), asegurando que reflejen fielmente las justificaciones teóricas de los principios SOLID. Desarrolló los métodos internos

de cálculo polimórfico y la integración de las colecciones de acompañantes.

- **Lautaro Vita:** Finalizó la programación de la capa de negocio, calibrando la fachada SistemaHotelFacade y asegurando la correcta integración de los patrones creacionales (HabitacionFactory) y la reestructuración dinámica del decorador con las solicitudes provenientes de la interfaz de usuario Swing.
- **Claudino Diaz:** Aseguró la persistencia real de los DAOs en la capa repo, optimizando las consultas SQL de HabitacionDAO, ReservaDAO y HuespedDAO conectadas a través de la clase única ConexionBD. Lideró el diseño de los escenarios de prueba funcionales en el entorno local.

11.2 Reflexión individual de cada integrante

Reflexión de Joaquín De Luca: A lo largo de este proyecto, el mayor aprendizaje fue comprender la importancia del Principio de Responsabilidad Única (SRP). Al principio, nuestra clase Reserva hacía demasiadas cosas y era muy rígida. Dividirla y aplicar polimorfismo para las tarifas me demostró cómo un buen diseño del dominio facilita enormemente la escalabilidad del sistema, haciendo que el código sea mucho más legible y menos propenso a errores.

Reflexión de Lautaro Vita: Mi mayor desafío fue lograr que la interfaz gráfica no quedara fuertemente acoplada a las reglas de negocio. Implementar el patrón Facade fue un punto de inflexión; me permitió entender cómo ocultar la complejidad del sistema detrás de una única puerta de entrada. Además, el uso de patrones creacionales como el Factory Method me hizo notar lo sencillo que resulta instanciar objetos complejos cuando las responsabilidades están correctamente delegadas.

Reflexión de Claudino Diaz: Desde el lado de la persistencia, este trabajo me ayudó a asimilar conceptos clave como la Alta Cohesión y la Fabricación Pura. Aislar completamente la base de datos a través de los DAO e implementar el patrón Singleton para la conexión no solo optimizó el uso de recursos, sino que me demostró en la práctica cómo proteger el modelo de dominio de los detalles técnicos de infraestructura, logrando un código verdaderamente modular.

12. Conclusión grupal:

El desarrollo de este trabajo práctico nos permitió comprender en la práctica el inmenso valor de la ingeniería de software y el diseño orientado a objetos. Comenzamos con una visión muy procedimental en la Fase 1, pero al ir iterando hacia las Fases 2 y 3, descubrimos que la correcta aplicación de los principios SOLID, patrones GRASP y patrones de diseño (como Facade, Decorator y Strategy) no son simples reglas teóricas, sino herramientas vitales que nos permitieron crear una arquitectura altamente cohesiva, desacoplada y preparada para futuras extensiones (como nuevos tipos de habitaciones o descuentos) sin tener que reescribir la lógica central. Logramos transformar una

problemática compleja en un sistema funcional, robusto y fácil de mantener.

13. Futuras mejoras

Gracias a que en esta Fase 3 logramos un código modular y bien desacoplado, pensamos algunas mejoras para más adelante incorporando nuevos patrones de diseño:

1. Portal de autogestión para huéspedes (Patrón Proxy)

Los clientes podrán loguearse en su propia cuenta, ver su estadía, reservar y pagar señas sin tener que pasar por el recepcionista. El Proxy actúa como intermediario que recorta los permisos según el rol: si el usuario logueado es un "Huésped", solo lo deja reservar y pagar, bloqueándole totalmente el acceso a las funciones administrativas exclusivas del personal.

2. Notificaciones en tiempo real (Patrón Observer)

Un sistema de alertas visuales que reaccione al instante cuando una reserva se confirma, se libera una habitación, o se actualizan los datos del dashboard del administrador. Las clases Reserva y Estadia actúan como Sujetos (Observables) y la bandeja de notificaciones y el dashboard son los Observadores. Si una reserva cambia de Pendiente a Confirmada, notifica automáticamente a todos los interesados sin que las clases dependan directamente unas de otras.

3. Exportar facturas a PDF (Patrón Adapter)

Al hacer el check-out, el sistema permitirá descargar el ticket final en un archivo PDF para enviárselo por mail al cliente. Se crea una interfaz propia ExportadorDocumento y una clase adaptadora iTextPDFAdapter que traduce los datos del sistema a lo que pide la librería externa (iText), evitando acoplar la clase Factura directamente a esa librería.